

a) Problem: Signal Convolution using Unit Impulse Signal

Logical Solution:

- $x(n) = \dots a_1\delta(n+2) + a_2\delta(n+1) + a_3\delta(n) + a_4\delta(n-1) + a_5\delta(n-2) + \dots$
- The output of LTI system of impulse signal is $h(n)$.
- $y(n) = \dots a_1h(n+2) + a_2h(n+1) + a_3h(n) + a_4h(n-1) + a_5h(n-2) + \dots$

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

x_n=np.array([2,0,2])
x_t=0
h_n=[1,1,1]
h_t=1
length=len(x_n)+len(h_n)-1
for i in range(length-len(h_n)+1):
    h_n.append(0)
h_n=np.array(h_n)
print(length,h_n)
```

5 [1 1 1 0 0 0]

Output-h(n)

```
# i) 2del(n)
t=np.array([ i-x_t for i in range(len(x_n))])
del_n=np.array([0 for i in range(len(x_n))])
del_t=x_t
for i in t:
    if i==0:
        del_n[i+x_t]=x_n[i]
    else:
        del_n[i]=0
del_n=np.array(del_n)
plt.grid(True)
plt.stem(t,del_n)
plt.show()
```

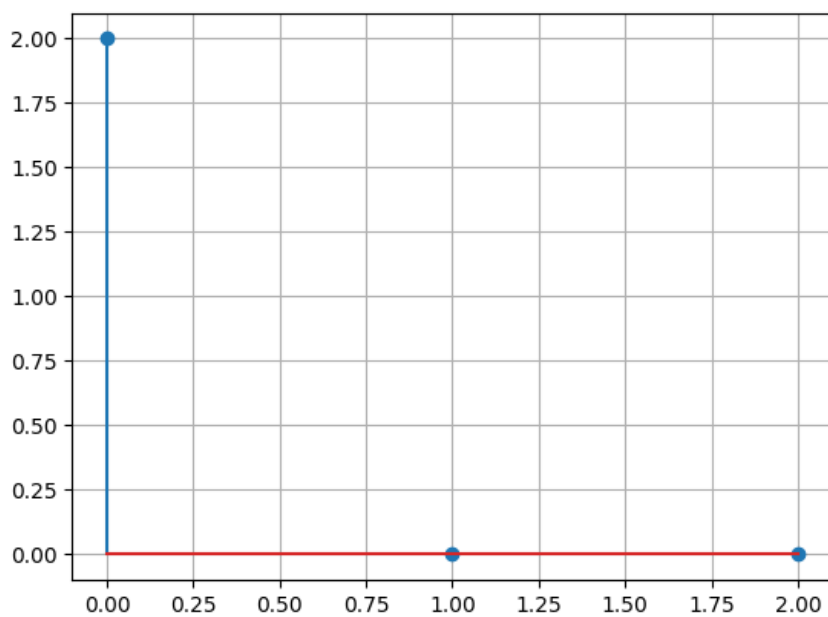


Figure-2 $\delta(n)$

```
# ii) 2h(n)
h1=h_n*del_n[0]
t=np.array([ i-h_t for i in range(len(h_n))])
plt.grid(True)
plt.stem(t,h1)
plt.show()
```

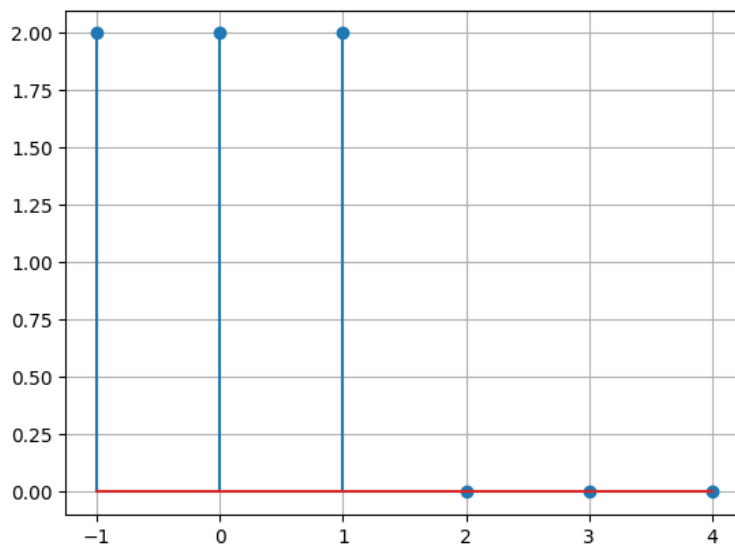


Figure-2h(n)

```
# iii)  $2\delta(n-2)$ 
```

```
t=np.array([ i-x_t for i in range(len(x_n))])
del_n=np.array([0 for i in range(len(x_n))])
print(del_n)
del_t=x_t
for i in t:
    if i==2:
        del_n[i+x_t]=x_n[i]
    else:
        del_n[i]=0
print(del_n)
del_n=np.array(del_n)
print(del_n,t)
plt.grid(True)
plt.stem(t,del_n)
plt.show()
```

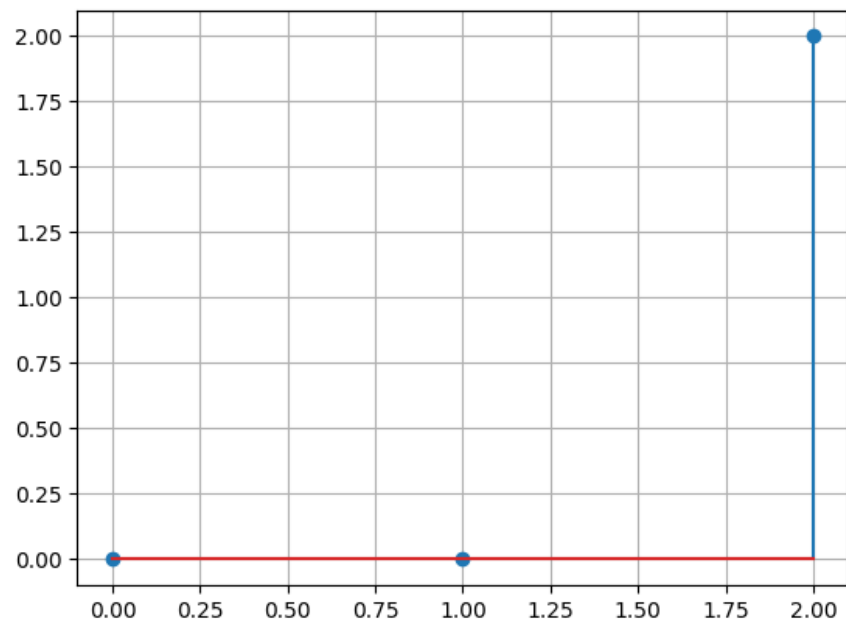


Figure-2 $\delta(n-2)$

```
# iv)  $2h(n-2)$ 
```

```
for i in range(2):
    h_n=h_n[:-1]
    h_n=np.insert(h_n,0,0)
```

```

h2=h_n*del_n[2]
print(h2)
t=np.array([ i-h_t for i in range(len(h_n))])
plt.grid(True)
plt.grid(True)
plt.stem(t,h2)
plt.show()

```

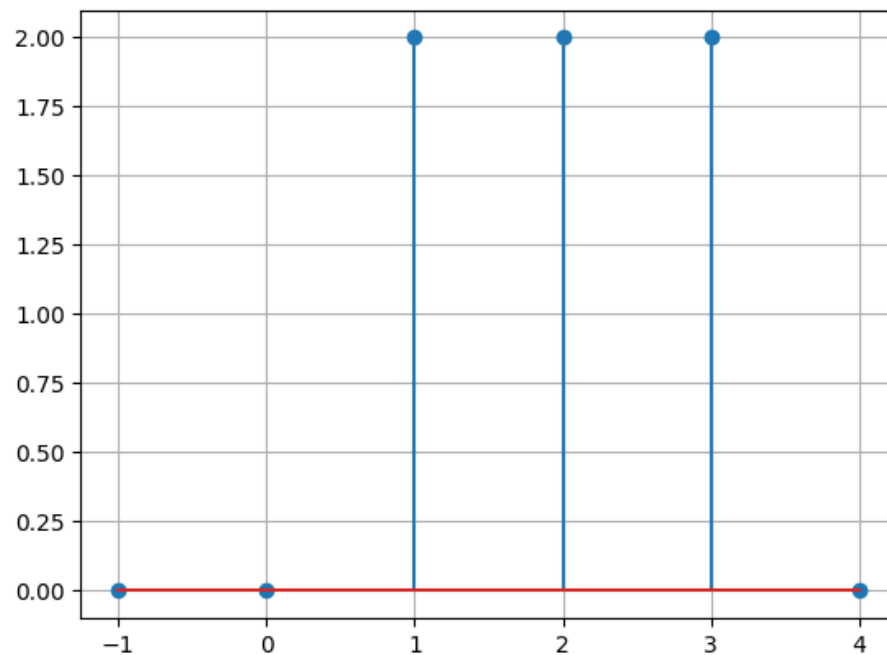


Figure-2h(n-2)

```

# v) h(n)+h(n-2)
y_n=h1+h2
print(y_n)
t=np.array([ i-h_t for i in range(len(h_n))])
plt.grid(True)
plt.stem(t,y_n)
plt.show()

```

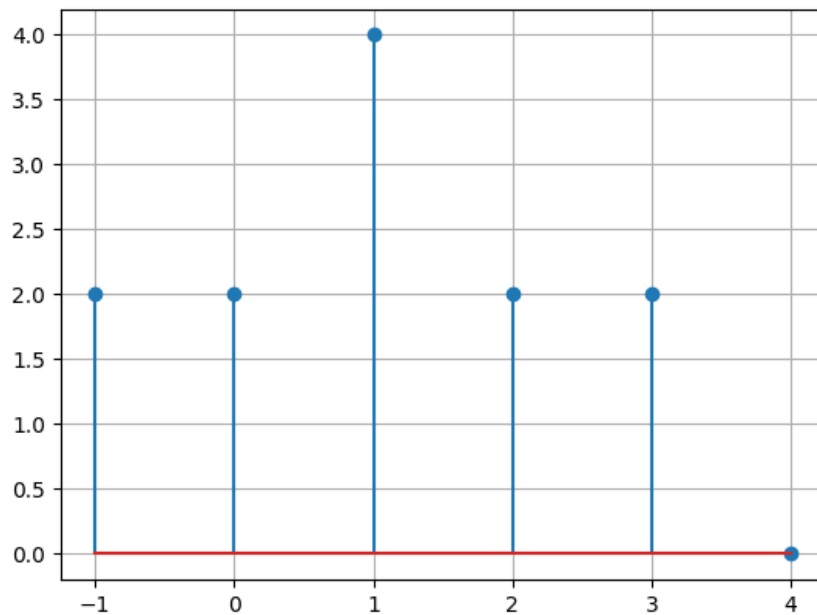


Figure-2h(n)+2h(n-2)

Discussion:

- Firstly, determine each unit impulse signal from the original signal
- Then send the impulse signal to the LTI system to get h(n), response signal

b) Problem: Signal Convolution using Summation Formula

Logical Solution:

- $$y(n) = \sum_{k=-\infty}^{\infty} x(n) * h(n - k)$$

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

x_n=np.array([1,2,3])
# x_n=np.array([1,2,4,8])
lenx=len(x_n)
x_t=0
h_n=np.array([4,5,6])
```

```

h_t=1
y_t=h_t
length=len(x_n)+len(h_n)-1
h_n=h_n[::-1]

for i in range(len(h_n)):
    x_n=np.insert(x_n,0,0)

for i in range(len(h_n)):
    x_n=np.append(x_n,0)

l=length-len(h_n)-h_t+1
for i in range(l):
    h_n=np.append(h_n,0)
    h_t-=1

for i in range(len(x_n)-len(h_n)):
    h_n=np.append(h_n,0)

print(h_n,x_n,h_t)

```

```
[6 5 4 0 0 0 0 0 0] [0 0 0 1 2 3 0 0 0] -1
```

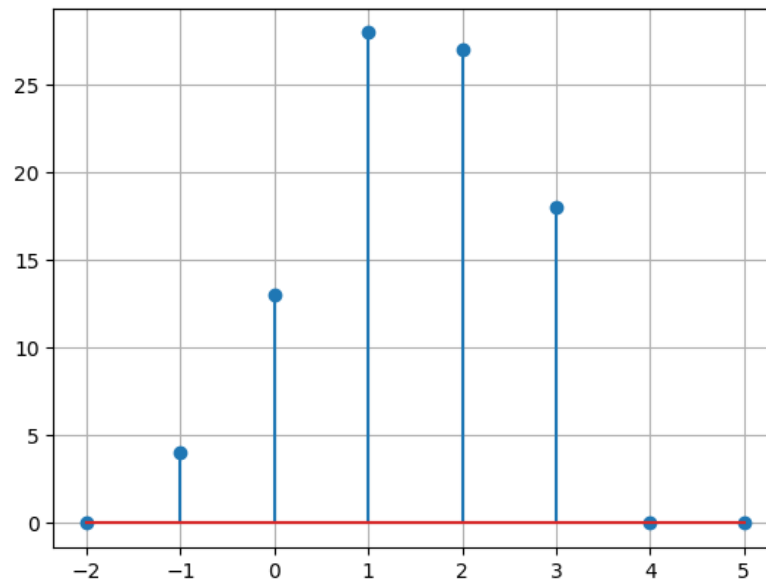
Output- $h(n)$, $x(n)$, $h(t)$

```

mn=min(h_t,x_t)
t=[]
y_n=[]
for i in range(mn,length+2):
    y_n.append(0)
    t.append(i-1)
    for j in range(len(h_n)):
        y_n[-1]+=x_n[j]*h_n[j]

    h_n=h_n[::-1]
    h_n=np.insert(h_n,0,0)
    h_t+=1
plt.stem(t,y_n)
plt.grid(True)
plt.show()

```



Output- Convolved Signal

Discussion:

- Firstly, I have reversed the response signal.
- Secondly, left shift that signal for proper calculation.
- Then shift that signal and multiply the values and sum it.

c) Problem: Convolution Using Built-in Function (0/1 Signal)

Logical Solution:

$$y(n) = \sum_{k=-\infty}^{\infty} x(n) * h(n - k)$$

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

x_n=[1,1,1,1,1,0,0,0,0,0,1,1,1,1,1,0,0,0,0,0]
h_n=[.2,.2,.2,.2,.2]

plt.stem(x_n)
plt.show()
```

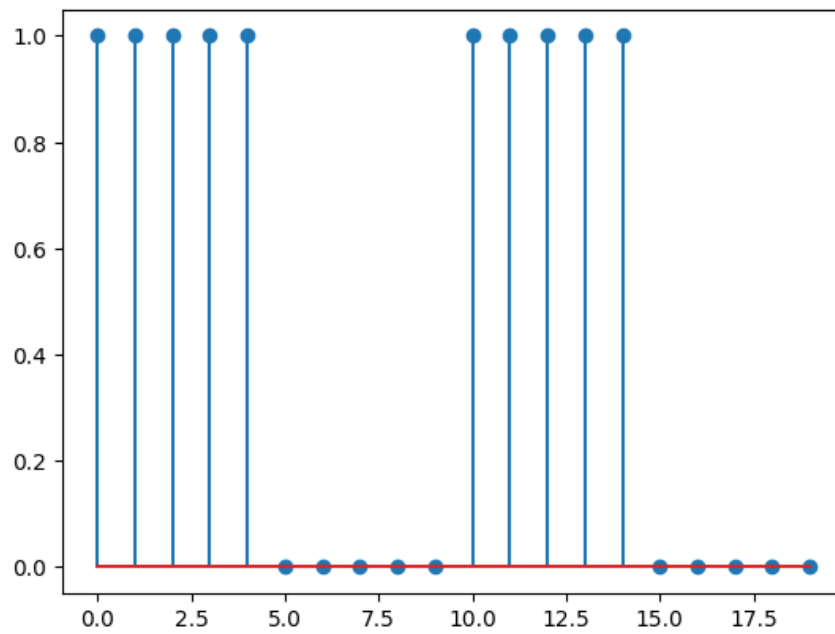
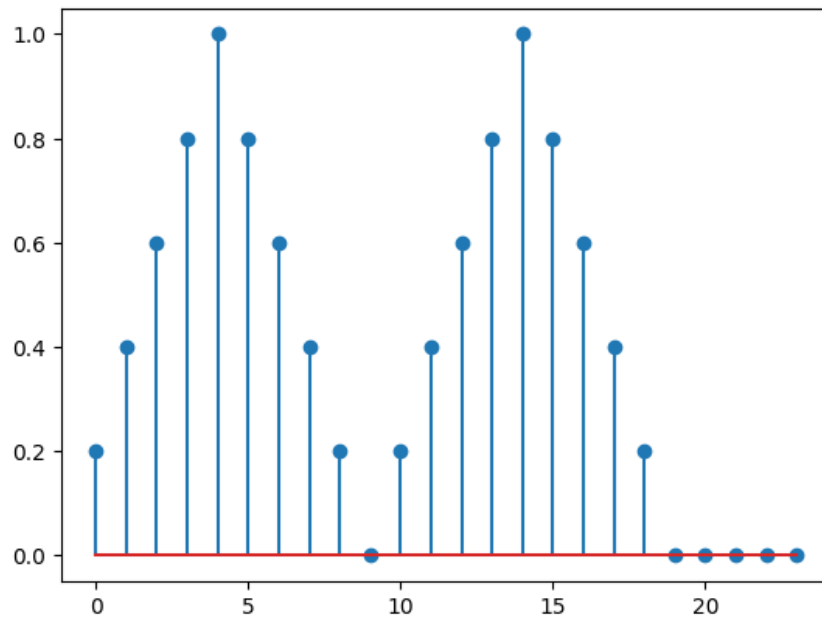


Fig- 0/1 Signal

```
y_n=np.convolve(x_n,h_n)
plt.stem(y_n)
plt.show()
```



Output- Convolved Signal

Discussion:

- Here I have used built-in numpy function which is `numpy.convolve(x,h)`.

d) Problem: Convolution Using Built-in Function (Sine Signal)**Logical Solution:**

- $y(n) = \sum_{k=-\infty}^{\infty} x(n) * h(n - k)$

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

n=np.linspace(-1,1,1000)
x_n=np.sin(2*np.pi*n)
plt.plot(x_n)
plt.show()
```

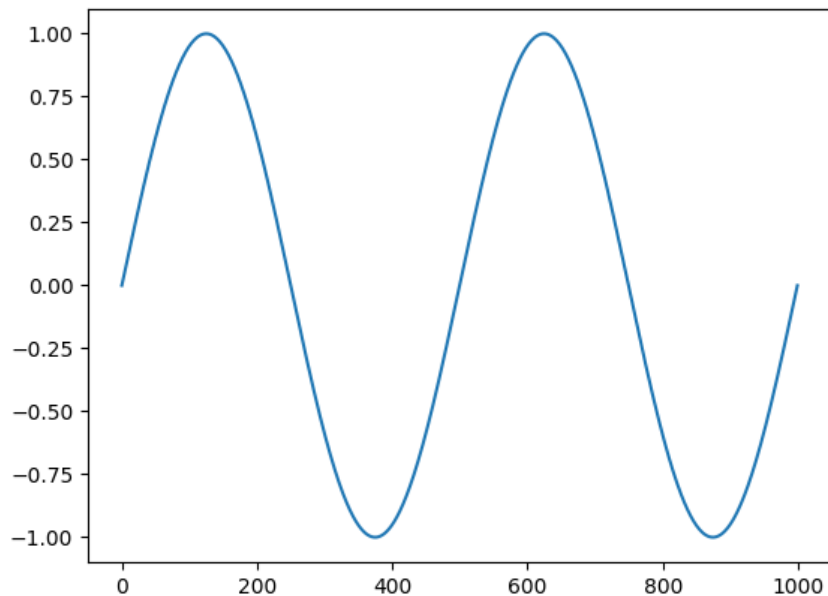
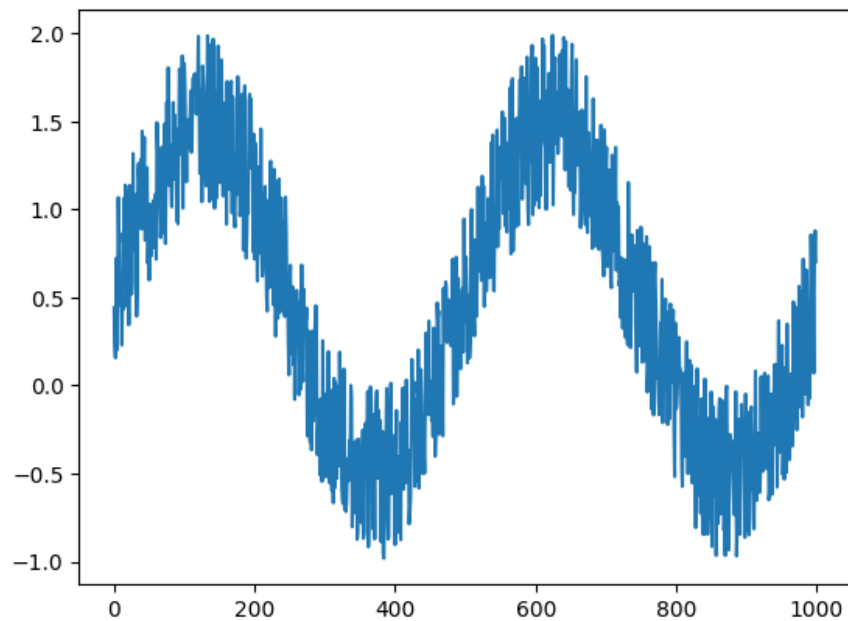


Fig- Sine Signal

```
noise=np.random.rand(len(n))
noisy=x_n+noise
plt.plot(noisy)
```

```
plt.show()
```



Output- Noisy Sine Signal

```
filtered=np.ones(100)/100  
after_filter=np.convolve(noisy,filtered)  
plt.plot(after_filter)  
plt.show()
```

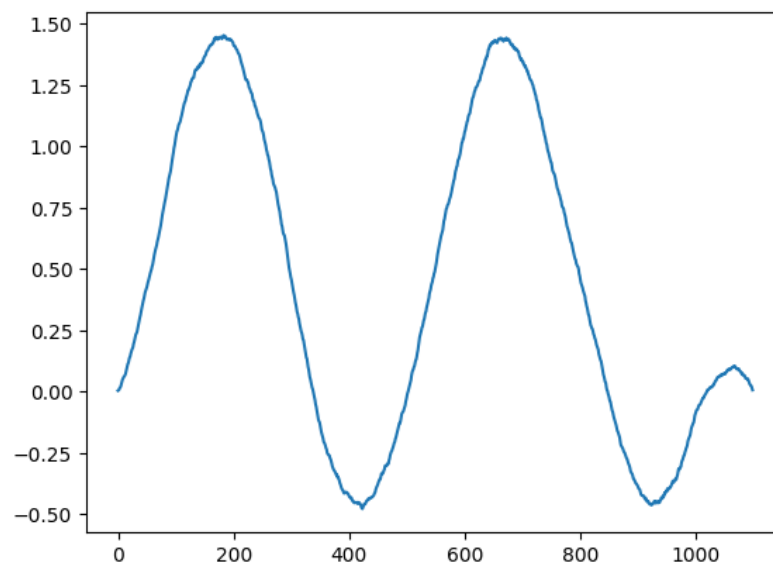


Fig- Filtered Signal

Discussion:

- After adding noise to original sinusoidal signal, pass it with the filtered value to the numpy built in function for convolution.